Operational Forensic Audit: Phase 3 Autopoietic Governance and the Policy of Truth (PoT) Implementation in the Stela Ecosystem

1. Executive Strategic Analysis: The Architectural Inflection of Phase 3

The contemporary software engineering landscape is currently undergoing a paradigm shift of unprecedented magnitude, transitioning from static, human-maintained repositories to dynamic, agentic ecosystems. The Stela platform, as evidenced by the forensic artifacts provided in the repository dump and the associated "AI Agent Constitution Review," stands at the vanguard of this transition. This report provides an exhaustive operational forensic audit of the platform's current state, specifically validating the execution of Dispatch Packet DP-OPS-0025, and analyzes the systemic implications of the newly established **Policy of Truth (PoT)** framework.

The analysis proceeds from the fundamental understanding that Stela is not merely a codebase but an implementation of **autopoietic systems theory** within a software development lifecycle.[1] Autopoiesis, a concept originally derived from biological cybernetics by Maturana and Varela, describes systems that are capable of reproducing and maintaining themselves within a topological boundary. In the context of Stela, this translates to a "Project Factory" architecture where the system's operating kernel (ops/) is decoupled from its variable payloads (projects/), creating a clean separation of concerns analogous to the "kernel-user space" division in monolithic operating systems.[1] This distinction is critical because, unlike allopoietic systems which produce something other than themselves (like a car assembly line producing cars), an autopoietic system's primary product is its own continued organization.

1.1 The Theoretical Framework of Autopoietic Software

To rigorously evaluate the success of the recent Phase 3 transition (DP-OPS-0025), one must first deconstruct the theoretical underpinnings that govern Stela's architecture. Stela represents a stark departure from traditional "Citizen Models" of AI interaction toward a "Mechanistic Doctrine".[1] This transition is not merely a change in tooling but a fundamental reorientation of the "Social Contract" between the human operator and the artificial agent.

1.1.1 The Mechanistic Doctrine vs. The Citizen Model

The "Citizen Model" of AI interaction anthropomorphizes artificial agents, treating them as trusted collaborators capable of discerning intent, understanding nuance, and "doing the right thing" amidst ambiguity. This model relies heavily on social contracts, implied context, and high-trust environments. While effective in human-centric teams where shared cultural context bridges gaps in documentation, the Citizen Model introduces catastrophic risks in agentic systems due to **"Epistemic Drift"**.[1] Epistemic Drift occurs when a corruption in the rule-defining mechanism—such as a template, a linter, or a prompt instruction—propagates exponentially through the system, creating a cascade of architectural divergence.

In contrast, the **"Mechanistic Doctrine,"** which Stela has formally adopted in Phase 3, posits that the system must physically and deterministically prevent the agent from performing unauthorized actions.[1] It rejects trust in favor of verification. Under this doctrine, an agent is not asked to follow a style guide; the system is engineered to crash if the style guide is violated. This shift is critical because, unlike human operators who may fatigue, rationalize shortcuts, or "bend the rules" for expediency, autonomous agents operate with relentless speed and absolute literalism. If a vulnerability exists in the "Project Factory" (specifically the ops/bin/project binary), every project instantiated by that factory inherits the vulnerability, leading to systemic fragility.[1]

The successful execution of DP-OPS-0025 has enshrined this doctrine within the Preamble of the PoT.md file, explicitly stating: "The system prevents action that violates policy. It does not request compliance".[1] This is a historic milestone for the repository, moving it from a state of "aspirational governance" to "deterministic enforcement."

1.1.2 From System Injection to Project Election

Phase 3 is operationally defined by the architectural pivot from "System Injection" to "Project Election".[1] In earlier iterations of the Stela platform, the system likely injected a global context into every agent session. While this ensured that agents had access to all available tools, it led to **"Context Haze"**—a degradation of AI reasoning capabilities caused by the saturation of irrelevant information. This phenomenon mirrors the "information overload" described in knowledge management theories, where the absence of a Single Source of Truth (SSOT) leads to cognitive paralysis.[1]

The new autopoietic model mandates that the global system (ops/) remains sterile, containing only the absolute minimum tools required for construction. The capabilities—Agents, Tasks, and Skills—are stored in the "Library Trinity" (docs/library/) and are only loaded into the runtime environment when explicitly "elected" by a project's STELA.md configuration file.[1] This reduces token overhead and sharpens agent focus, but it introduces a new dependency: **Referential Integrity**. If a project elects a capability that does not exist (a "Dead Pointer"), the autopoietic process fails. Thus, the verification of DP-OPS-0025 centers on whether the system has successfully implemented the linters and guards necessary to enforce this

integrity, ensuring that the "machine specification" remains valid at all times.[1]

1.2 The Strategic Necessity of DP-OPS-0025

Dispatch Packet DP-OPS-0025 was designed as a strategic hardening operation to close the "audit gaps" exposed by the transition to Phase 3. The "AI Agent Constitution Review" identified critical vectors of fragility—specifically the "Scattered Truth" problem—that threatened to undermine the entire autopoietic experiment.[1]

Prior to this packet, the "Constitutional" logic of Stela was distributed across at least four primary surfaces: TRUTH.md (Constitution), AGENTS.md (Behavior), SECURITY.md (Policy), and CONTRIBUTING.md (Workflow).[1] This fragmentation forced an agent to maintain four separate context windows to understand its own constraints. In an autopoietic system, where the system must "understand itself" to reproduce, this complexity increased the error rate of self-replication.

The strategic intent of DP-OPS-0025 was to consolidate these surfaces into a monolithic **Policy of Truth (PoT.md)**. This document serves as the "Supreme Statute" of the Stela repository, superseding all other governance documents. By centralizing the physical laws (TRUTH), behavioral standards (AGENTS), and security invariants (SECURITY) into a single file, DP-OPS-0025 has created a unified "Truth Layer" that eliminates ambiguity and redundancy.[1]

The successful execution of this packet is not merely a maintenance task; it is the foundational requirement for the system's survival as an autonomous entity. Without these controls, the system cannot safely self-replicate, and the vision of a "Technium" that behaves as a superorganism remains out of reach. The forensic analysis that follows confirms that "History has been made": the repository has achieved a state of self-contained, verifiable constitutional integrity.

---

2. Forensic Verification of DP-OPS-0025 Execution

This section presents a forensic audit of the repository state (HEAD 77cd351fc) to determine if the execution of DP-OPS-0025 met its strategic objectives. The analysis relies on the "State of Play" (SoP.md) ledger, the "Policy of Truth" (PoT.md), and the file manifest provided in the repository dump.[1]

2.1 The Evidence of Execution: Analyzing the Ledger

The primary artifact for verification is SoP.md (State of Play). In the Stela governance model, SoP.md serves as the immutable history ledger; a task is legally defined as "complete" only when it is recorded here. This mirrors the function of a blockchain or a secure audit log in

financial systems, where the record of the transaction is the transaction itself.[1]

Audit Finding: The SoP.md file contains a specific entry for 2026-02-05 - DP-OPS-0025. This entry explicitly logs the following actions [1]:

- **Purpose:** "Consolidate governance surfaces into PoT and resolve jurisdiction ambiguity."
- **Shipped Artifacts:**
  - "Created PoT.md consolidating TRUTH, AGENTS, SECURITY, and CONTRIBUTING with the Mechanistic Doctrine, Sheriff enforcement, and Multi-verse jurisdiction."
  - "Updated governance pointers, manifests, and ops/bin tooling to reference PoT.md."
  - "Tombstoned SECURITY.md and CONTRIBUTING.md."
  - "Archived legacy TRUTH.md and AGENTS.md into storage/archives/root/ with date suffixes."
- **Verification Steps:** The log records the successful execution of ./ops/bin/dump, tools/lint/context.sh, tools/lint/truth.sh, and ops/lib/scripts/skill.sh harvest.

Conclusion: The presence of this detailed entry in the canonical ledger provides *prima facie* evidence that the governance requirements for recording the work have been met. The system acknowledges the change, satisfying the "Equilibrium Maintenance" directive of the PoT. This adherence to the ledger prevents the "hidden drift" often seen in systems where code changes occur without corresponding governance updates.

2.2 Deep Dive: The Monolithic Constitution (PoT.md)

The creation of PoT.md is the centerpiece of DP-OPS-0025. A structural analysis of the file confirms that it effectively merges the disparate governance logic into a coherent whole.

Original Document	Migrated Section in PoT.md	Status	Analysis
TRUTH.md	Section 1: The Constitution	**Migrated**	Defines the "Filing Doctrine" (ops/=Run, docs/=Explain) and Axioms. This establishes the "physical laws" of the repo.[1]
AGENTS.md	Section 4: Staffing & Logic	**Migrated**	Defines the Staffing Protocol (Operator/Integrato

			r/Contractor) and Behavioral Logic (No contractions). This establishes the "behavioral laws".[1]
SECURITY.md	Section 5: Workflow & Security	**Migrated**	Defines Secrets Management and AI Usage Policy. This integrates security as a subset of governance rather than a separate silo.[1]
CONTRIBUTING.md	Section 5: Workflow & Security	**Migrated**	Defines "Non-Negotiables" (No push to main, Work on work/* branches). This codifies workflow as law.[1]

Preamble Analysis: The Preamble of PoT.md explicitly defines the **"Mechanistic Doctrine"**: "The system prevents action that violates policy. It does not request compliance.".[1] This is a critical addition that was not present in the legacy TRUTH.md. It sets the interpretive framework for the entire document: these are not guidelines; they are constraints.

Jurisdictional Clarity: Section 3 of PoT.md introduces the **"Multi-verse"** model, distinguishing between "The Crown" (Platform) and "The Colonies" (Projects).[1]

- **The Crown:** "Absolute PoT enforcement. No contractions. Strict formatting." This ensures the kernel remains stable and unambiguous.
- **The Colonies:** "Structural adherence to platform governance... Local law applies to project content." This resolves the ambiguity of whether user projects must adhere to the strict style guides of the platform.

This separation is vital for scalability. It allows the platform to be rigorous (ops/) while the payloads (projects/) can be flexible, enabling innovation at the edge without compromising stability at the core.

2.3 The "Tombstone" Strategy

Forensic analysis of the file dump reveals that SECURITY.md and CONTRIBUTING.md still exist but have been modified to serve as "Tombstones".[1]

- **Content of SECURITY.md:** # SECURITY.md \n Migrated to PoT.md. See PoT.md for active policy.
- **Content of CONTRIBUTING.md:** # CONTRIBUTING.md \n Migrated to PoT.md. See PoT.md for active policy.

This strategy is superior to deletion for two reasons. First, it preserves the file history in git, allowing for provenance tracking. Second, it serves as a navigational aid for human operators or legacy tools that may hard-code lookups for these standard filenames. It prevents "404 Governance," where an agent might assume there are no security rules because SECURITY.md is missing. Instead, it is explicitly redirected to the new SSOT.

2.4 Linter Alignment and Truth Enforcement

A governance document is only as good as its enforcement mechanism. The SoP.md log for DP-OPS-0025 notes: "Updated tools/lint/truth.sh to scan PoT.md and refreshed context bundle inputs".[1]

The source code for tools/lint/truth.sh confirms this alignment. It defines root_files to include PoT.md and explicitly scans for forbidden spellings (like "Stella" instead of "Stela").[1] This confirms that the "Sheriff" (the enforcement automation) has been successfully deputized to uphold the new constitution. If this update had been missed, the system would have entered a state of "Lawlessness," where the rules existed on paper (PoT.md) but the police (truth.sh) were ignoring them. The successful synchronization of Law and Enforcement is a key indicator of the audit's success.

---

3. The Economy of Truth: Assessing Context Efficiency

The **"Economy of Truth"** is the operational philosophy that governs how context is managed within the Stela ecosystem. In the age of Large Language Models (LLMs) with finite context windows (even large ones), context is a scarce economic resource. "Context Haze"—the dilution of relevant signal by irrelevant noise—is a primary cause of agent hallucination and failure.

3.1 The Architecture of llms.txt

The llms.txt file is the machine-readable entry point for the repository. Its role is to provide a high-level map of the territory without duplicating the territory itself.

Forensic Analysis: The content of llms.txt [1] adheres strictly to the "Pointer-first" constitution. It does not contain long prose; instead, it lists the canonical files:

- **Context Bundles:** Points to llms-small.txt and llms-full.txt.
- **Governance:** Points to PoT.md.
- **State:** Points to SoP.md and TASK.md.
- **Operations:** Points to docs/library/MANUAL.md.

This structure minimizes entropy. An agent reading llms.txt consumes very few tokens but gains a complete map of where to find detailed information. This prevents the "Context Haze" that would occur if llms.txt attempted to summarize everything inline.

3.2 Context Bundling: The llms Binary

The ops/bin/llms binary is the engine of the Economy of Truth. It mechanically assembles the context bundles (llms-small.txt and llms-full.txt) based on the manifest defined in ops/lib/manifests/CONTEXT.md.[1]

- **Small Bundle:** Includes PoT.md, SoP.md (head 10), TASK.md, and llms.txt. This is the "Survival Kit." It provides the agent with the Laws, the History, and the Current Objective.
- **Full Bundle:** Adds docs/library/ and README.md. This is the "Reference Library."

Critical Feature: The Context Hazard Guard. The ops/bin/llms script includes a specific function validate_bundle_paths that checks for "Context Hazards".[1]

Bash

```
local hazards=("docs/library/agents" "docs/library/tasks" "docs/library/skills")
#...
if [[ "$path" == "$hazard"* ]]; then
    die "Context hazard: ${bundle_name} includes ${path}"
fi
```

This code proves that the system *actively prevents* the inclusion of the heavy library files (agents, tasks, skills) in the global context manifest. These files are designated as "Just-In-Time" (JIT) resources—they should only be loaded when an agent explicitly "elects" them for a specific task. This mechanism physically enforces the "Economy of Truth" by preventing the accidental pollution of the global context with massive library dumps.

3.3 The Pruning Mechanism

The ops/bin/prune script complements the bundling system by maintaining the hygiene of the SoP.md ledger.[1] The audit log confirms its execution during DP-OPS-0025.

Mechanism: The script checks the number of entries in SoP.md. If the count exceeds a

threshold (30), it moves the older entries to an archive file in storage/archives/root/.

Strategic Value: This sliding window approach ensures that the "State of Play" remains relevant. An agent needs to know what happened last week to maintain continuity, but it does not need to load the entire history of the repository since its inception. By pruning the tail of the log, the system keeps the "Small Bundle" actually small, preserving token bandwidth for the active task.

---

4. Operational Tooling Analysis: The "Ops" Layer

The ops/ directory is the "Kernel" of the Stela system. The binaries within it are the executables that enforce the Mechanistic Doctrine. A line-by-line analysis of the key scripts reveals a robust security posture aligned with Phase 3 requirements.

4.1 ops/bin/open: The Session Gatekeeper

The ops/bin/open script is the first command run in any session. It generates the "OPEN PROMPT" that boots the agent.[1]

Code Analysis [1]:

- **Freshness Gate:** branch="$(git rev-parse --abbrev-ref HEAD)". The script captures the current git state, grounding the agent in time.
- **Porcelain Check:** `porcelain_raw="$(git status --porcelain |

| true)"`. It checks if the working tree is dirty. If it is, it captures the diff summary. This is a vital "Anti-Drift" mechanism. It prevents an agent from starting work on top of uncommitted changes, which would lead to a "Schrodinger's Repo" state where the file system and the git index disagree.

- **Role Contract:** The output prompt explicitly defines the roles: "Integrator (operator): Owns scope... Contractor (assistant): Executes tasks." This reinforces the hierarchy defined in PoT.md.
- **Context Loading:** It explicitly lists the required context (PoT, SoP, TASK), ensuring the agent acknowledges the "Must Load" canon before proceeding.

4.2 ops/bin/dump: The State Serializer

The ops/bin/dump script provides the "sensory input" for the agent. It serializes the repository state into a text stream.[1]

Robustness:

- **Scoping:** The script supports --scope=platform (default) and --scope=full. The "Platform" scope excludes projects/, focusing the agent on the kernel. This supports the

"Multi-verse" model by allowing maintenance of the Crown without distraction from the Colonies.
- **Deterministic Listing:** It uses git ls-files to select files. This ensures that the agent only sees what is tracked by git, ignoring local trash or untracked artifacts in storage/. This enforces the "Single Source of Truth" principle—if it's not in git, it doesn't exist for the agent.

4.3 ops/bin/project: The Factory Interface

The ops/bin/project script is the interface for creating and managing payloads. It serves as the "Factory Floor" manager.

Phase 3 Alignment: The script sources ops/lib/scripts/project.sh for its logic. This separation of interface (binary) and logic (library) is a standard architectural best practice. The script commands (init, work, status) provide a structured workflow for interacting with the "Colonies," ensuring that projects are created with the correct scaffold and registered in the PROJECTS.md manifest.[1]

4.4 ops/lib/scripts/project.sh: The Hardened Library

This library file contains the critical validation logic. The forensic audit of DP-OPS-0024 (referenced in the input) highlighted the hardening of this file.

Security Analysis [1]:

- **Regex Validation:** if [[! "$value" =~ ^[a-zA-Z0-9_-]+$]]; then project_die "Invalid ${label}: ${value}". This strict regex prevents shell injection and path traversal characters. A user cannot name a project ../../ops to overwrite the kernel.
- **Realpath Resolution:** resolved="$(realpath "$path" 2>/dev/null)". The script resolves all paths to their absolute location before acting. This defeats symlink attacks and ensures that operations are strictly confined to the intended directory.
- **Registry Enforcement:** The script validates against ops/lib/manifests/PROJECTS.md. A project cannot be "worked on" unless it is registered. This prevents "Ghost Projects" (directories that exist but are not tracked) and ensures complete visibility of the system state.

---

5. Structural Hygiene and Directory Auditing

The user query specifically asks about the necessity of docs/ops/ and docs/security/. Based on the "Filing Doctrine" defined in PoT.md ("ops/ = Run... docs/ = Explain"), we can perform a rigorous structural audit.

5.1 The Redundancy of docs/security/

Current State: The dump shows docs/security/README.md exists.[1] Its content is likely a pointer to the root SECURITY.md.

Analysis: The PoT.md file (Section 5.4) now contains the comprehensive "Security Policy (Stela)," including Secrets Management, AI Usage Policy, and Vulnerability Reporting.[1] The legacy SECURITY.md is tombstoned. Therefore, docs/security/README.md is a "Pointer to a Pointer." In information theory, this is inefficient encoding. In operations, it is "Click Fatigue."

Verdict: REMOVE.

The "Economy of Truth" dictates that we minimize the number of hops to reach the SSOT. Since PoT.md is the single source of security policy, the docs/security/ directory adds no value. It creates the illusion of a separate security documentation silo, which contradicts the integrated governance model of Phase 3. Specific operational security guides (e.g., "How to Rotate AWS Keys") should live in docs/library/skills/ as executable skills, not in a passive docs/security/ folder.

5.2 The Ambiguity of docs/ops/

Current State: docs/ops/INDEX.md exists.[1] It lists the operational binaries.

Analysis:

The separation of ops/ (Execution) and docs/ (Explanation) is architecturally sound. However, the current docs/ops/INDEX.md appears to merely duplicate the list of commands found in docs/library/MANUAL.md. Duplication is a violation of SSOT.

Verdict: CONSOLIDATE & REFACTOR.

We should not simply delete docs/ops/ because there is a valid need for deep architectural documentation of the kernel (e.g., "The Design Philosophy of the Context Bundler"). However, an INDEX.md that acts as a weak mirror of the Manual is clutter.

Recommendation:

1. **Rename** docs/ops/INDEX.md to docs/ops/README.md.
2. **Refactor** its content to focus on *Architecture* (Why the scripts work this way) rather than *Usage* (How to run them). Usage belongs in MANUAL.md.
3. If no deep architectural docs exist yet, consolidate the index into MANUAL.md and delete the directory to reduce surface area until such docs are written.

---

6. Semantic Refactoring: The "Sheriff" and "Colonies"

The user prompt explicitly states: "Leave the concepts in tact just naming conventions like 'Colonies' or 'Sheriff' and there are a few others - they need to go." This directive requires a

semantic refactoring of the PoT to align with a professional, enterprise-grade nomenclature while preserving the underlying autopoietic concepts.

6.1 "The Sheriff" (Enforcement Mechanism)

Current Concept: Defined in PoT.md Section 2 as "The Integrator plus Automation" with the mandate to "Kill processes".[1] **Critique:** "Sheriff" relies on a "Wild West" frontier metaphor. It implies individual heroism and a certain degree of lawlessness that requires a gun-toting enforcer. This contradicts the "Mechanistic Doctrine," which aims for deterministic, impersonal, system-level enforcement.

Proposed Replacement: "The Governance Engine"

- **Rationale:** "Governance Engine" implies a tireless, automated, and integrated subsystem. It sounds like a component of a kernel, not a person.
- **The Enforcer Pattern:** We can also use **"The Sentinel"** for the monitoring aspect (linters) and **"The Executive"** for the action aspect (killing processes). However, **"Governance Engine"** covers both.
- **Refactored Text:** "The **Governance Engine** (Integrator + Automation) enforces the PoT. Mandate: Halt execution, Reject PRs..."

6.2 "The Colonies" (Project Jurisdiction)

Current Concept: Defined in PoT.md Section 3.2 as projects/. Projects adhere to platform structure but have local laws.[1] **Critique:** "Colonies" carries significant historical and geopolitical baggage (imperialism, extraction, subservience). In a software context, it is imprecise; projects are not settlements, they are workloads. It implies a "center-periphery" dynamic that is more political than technical.

Proposed Replacement: "The Federation" (or "Federated Projects")

- **Rationale:** "Federation" implies a union of semi-autonomous entities under a central constitution (PoT). This perfectly captures the "Multi-verse" model where projects respect the federal law (Platform) but maintain state rights (Local Style).
- **Alternative: "User Space"**. This aligns with the "Kernel" (ops/) analogy.
- **Recommendation: "The Federation."** It preserves the governance nuance better than "User Space."

6.3 "The Crown" (Platform Jurisdiction)

Current Concept: Defined in PoT.md Section 3.1 as ops/, docs/, tools/. "Absolute PoT enforcement".[1] **Critique:** "The Crown" pairs with "Colonies" in the imperial metaphor. If "Colonies" goes, "Crown" must go to maintain semantic consistency.

Proposed Replacement: "The Core" (or "The Platform Core")

- **Rationale:** "The Core" is standard systems terminology. It denotes the immutable center of the system without the monarchical implications.
- **Refactored Text:** "**The Core** (Platform Scope): ops/, docs/, tools/... Absolute PoT enforcement."

6.4 "Multi-verse" (Jurisdiction Model)

Current Concept: The overarching model of split jurisdiction.

Critique: "Multi-verse" is slightly sci-fi and implies completely disconnected realities.

Proposed Replacement: "Tiered Jurisdiction" or **"Federal Architecture."**

- **Recommendation: "Tiered Jurisdiction."** It is precise and descriptive.

6.5 Summary of Semantic Refactoring

Current Term	Proposed Term	Concept Preserved
The Sheriff	**The Governance Engine**	Automated enforcement of invariants.
The Colonies	**The Federation**	Semi-autonomous project payloads.
The Crown	**The Core**	Immutable platform kernel.
Multi-verse	**Tiered Jurisdiction**	Split governance model.

---

7. Strategic Roadmap: Recommendations for DP-OPS-0026

Based on the forensic audit and the directive for semantic professionalization, the next Dispatch Packet (DP-OPS-0026) must execute a targeted refactoring of the repository's language and structure.

7.1 Scope of DP-OPS-0026

1. **Semantic Refactor (PoT.md):**
    - Find and replace all instances of "Sheriff" with "Governance Engine."
    - Find and replace all instances of "Crown" with "The Core."
    - Find and replace all instances of "Colonies" with "The Federation."

    - Update the "Jurisdiction" section to use "Tiered Jurisdiction" instead of "Multi-verse."
2. **Structural Pruning:**
    - **Delete** docs/security/. Move any unique content (if it exists) to docs/library/.
    - **Refactor** docs/ops/. Rename INDEX.md to README.md and rewrite it to describe the *architecture* of the ops layer. If unable to add value, consolidate into MANUAL.md and delete the directory.
3. **Linter Hardening:**
    - **Update tools/lint/truth.sh:** Add "Sheriff", "Colonies", and "Crown" to the forbidden_spellings array.[1] This is the "Mechanistic Doctrine" in action—the system will henceforth physically prevent the re-introduction of the old metaphors.
    - **Fix Markdownlint:** The audit log noted "style FAIL: markdownlint missing".[1] DP-OPS-0026 must ensure markdownlint is installed in the CI environment or the linter script (tools/lint/style.sh) is robust enough to handle its absence (or fail hard if strictness is required). The PoT mandates verification; a missing tool is a security gap that must be closed.

8. Conclusion

The execution of DP-OPS-0025 was a decisive operational success. It established a **Single Source of Truth (PoT.md)**, effectively tombstoned legacy policy files, and secured the "Project Factory" architecture against context drift. The system now operates under a **Mechanistic Doctrine** where compliance is enforced by code (truth.sh, project.sh) rather than requested of users. The "Economy of Truth" is functioning efficiently, reducing cognitive load through smart context bundling.

However, the "Sheriff/Colonies" nomenclature represents a semantic debt—a remnant of the system's developmental phase. To complete the transition to a professional, enterprise-grade autopoietic system, DP-OPS-0026 must execute the semantic refactoring proposed in this report. This will align the system's language with its architecture: precise, neutral, and ruthlessly efficient.

Final Verdict: The Stela repository has made history by successfully implementing a self-enforcing, autopoietic constitution. With the upcoming semantic refactoring, it will achieve a state of "Professional Autopoiesis," ready for indefinite scaling.

Status: READY FOR DP-OPS-0026

Works cited

1. AI Constitution and Dispatch Packet Draft.pdf